EXERCICE PRATIQUE

CI/CD avec GitHub Actions

Automatiser, tester et sécuriser le déploiement de son application

NIVEAU	PLATEFORME
Débutant	GitHub
TECHNOLOGIE	**DURÉE**
Node.js	3–4 heures

📋 Sommaire

1. C'est quoi le CI/CD ? (les concepts de base)
2. C'est quoi un pipeline ? (explication détaillée)
3. Présentation du projet fil rouge
4. Étape 1 — Créer le dépôt GitHub
5. Étape 2 — Découvrir l'application
6. Étape 3 — Comprendre la structure d'un pipeline GitHub Actions
7. Étape 4 — Observer le premier pipeline CI
8. Étape 5 — Provoquer un échec (et comprendre pourquoi c'est utile)
9. Étape 6 — Sécurité : les secrets GitHub
10. Étape 7 — Protection des branches
11. Étape 8 — Le pipeline CD et les environnements
12. Étape 9 — Sécurité des images Docker avec Trivy
13. Étape 10 — Analyse du code source avec CodeQL (SAST)
14. Étape 11 — Scénario complet : la Pull Request
15. Récapitulatif : les 7 règles d'or de la sécurité CI/CD
16. Exercices bonus

1. C'est quoi le CI/CD ?

Avant de commencer à manipuler des outils, il faut bien comprendre le problème que CI/CD résout. Prenons un exemple concret.

Le problème sans CI/CD

Imaginez une équipe de 4 développeurs sur le même projet. Chacun travaille dans son coin pendant 2 semaines, puis le vendredi après-midi, tout le monde fusionne son code.

> 💥 **Ce qui se passe en réalité**
> Les codes ne fonctionnent plus ensemble. Des tests cassent. Des fonctionnalités se régressent mutuellement. Le vendredi soir vire au désastre. On appelle ça **l'enfer de la fusion** (merge hell).

La solution : l'Intégration Continue (CI)

L'**Intégration Continue** (Continuous Integration) consiste à vérifier automatiquement chaque modification de code dès qu'elle est envoyée.
Concrètement : à chaque `git push`, une machine virtuelle se lance et exécute une série de contrôles :

- Est-ce que le code est bien écrit (respect des conventions) ?
- Est-ce que les tests passent ?
- Est-ce qu'on n'a pas introduit de faille de sécurité ?

Si un contrôle échoue, le développeur est notifié immédiatement, sur sa propre modification. Plus de surprise le vendredi.

Le Déploiement Continu (CD)

Le **Déploiement Continu** (Continuous Delivery) va plus loin : une fois que tous les contrôles sont passés, le code est automatiquement livré vers un environnement (staging, production).

```
Développeur fait un git push │ ▼ ┌──────────────────────────────┐ │ Pipeline CI
│ │ 1. Analyse du code (lint) │ │ 2. Tests unitaires │ │ 3. Audit de sécurité │ │
4. Construction image Docker │ └──────────────────────────────┘ │ (si tout est
```

```
OK ✅) ▼ ┌──────────────────────────────┐ │ Pipeline CD │ │ 1. Publication de
l'image │ │ 2. Déploiement en staging │ │ 3. Approbation manuelle │ │ 4.
Déploiement en production │ └──────────────────────────────┘ │ ▼ Application en
ligne !
```

> 💡 **Résumé simple**
>
> **CI** = filet de sécurité qui attrape les erreurs avant qu'elles n'arrivent en production.
> **CD** = automatisation de la livraison une fois les contrôles passés.

2. C'est quoi un pipeline ?

Un **pipeline** est une chaîne d'étapes automatisées qui s'exécutent dans un ordre précis, comme une chaîne de montage dans une usine.

L'analogie de la chaîne de montage

Dans une usine automobile :

1. On assemble le châssis
2. On installe le moteur
3. On peint la carrosserie
4. On contrôle la qualité
5. On livre la voiture

Si l'étape 2 échoue (moteur défectueux), on s'arrête là. On ne peint pas une voiture sans moteur.
Un pipeline CI/CD fonctionne exactement pareil :

```
┌──────────┐ ┌──────────┐ ┌──────────┐ ┌──────────┐ │ ÉTAPE 1 │──▶│ ÉTAPE 2
│──▶│ ÉTAPE 3 │──▶│ ÉTAPE 4 │ │ Lint │ │ Tests │ │ Sécu │ │ Build │ └──────────┘
└──────────┘ └──────────┘ └──────────┘ ✅ ✅ ✅ ✅ │ ▼ DÉPLOIEMENT
```

```
┌──────────┐ ┌──────────┐ │ ÉTAPE 1 │──▶│ ÉTAPE 2 │ ← ÉCHEC ici │ Lint │ │ Tests
│ └──────────┘ └──────────┘ ✅ ❌ │ └──▶ STOP. Les étapes suivantes ne s'exécutent
```

```
  pas. Le développeur est notifié.
```

Les composants d'un pipeline GitHub Actions

Dans GitHub Actions, un pipeline s'appelle un **workflow**. Il est défini dans un fichier `.yml` dans le dossier `.github/workflows/`.

Terme	Définition	Exemple
Workflow	Le pipeline complet (fichier .yml)	`ci.yml`
Trigger (déclencheur)	L'événement qui lance le workflow	`git push`, `pull request`
Job	Groupe d'étapes exécutées sur une même machine virtuelle	`lint`, `test`, `build`
Step (étape)	Une commande ou une action dans un job	`npm install`, `npm test`
Action	Bloc de code réutilisable (comme un plugin)	`actions/checkout@v4`
Runner	La machine virtuelle qui exécute le job	`ubuntu-latest`

Structure visuelle d'un fichier workflow

```yaml
name: CI - Intégration Continue    # Nom affiché dans l'onglet Actions

on:                                # Déclencheurs
  push:
    branches: [ "**" ]             # Sur TOUT push sur TOUTE branche
  pull_request:
    branches: [ main ]             # Sur toute PR vers main

jobs:
  lint:                            # Nom du 1er job
    runs-on: ubuntu-latest         # Machine virtuelle Ubuntu
    steps:
      - name: Récupérer le code
        uses: actions/checkout@v4      # Action GitHub (plugin)

      - name: Installer Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'

      - name: Lancer le lint
        run: npm run lint              # Commande shell classique
        working-directory: app
```

💡 **Ce qui se passe concrètement**

Chaque fois que vous faites un `git push`, GitHub loue une machine virtuelle Ubuntu quelque part dans le cloud, exécute votre pipeline (ça prend 1 à 5 minutes selon la complexité), puis vous affiche un ✅ ou ❌. La machine est ensuite détruite. Vous ne payez rien (quota gratuit généreux sur les repos publics).

Jobs en parallèle vs en série

Les jobs peuvent s'exécuter en parallèle (en même temps) ou en série (l'un après l'autre).
Dans notre pipeline CI, on a configuré ceci :

```
lint ──────────────
```

Cela signifie que `test` et `security-audit` tournent en même temps (gain de temps), mais `build-docker` n'attend que si les deux ont réussi.

3. Présentation du projet fil rouge

Pour cet exercice, vous allez travailler sur une **API REST** très simple : une calculatrice.

Pourquoi une calculatrice ?

L'application est volontairement simple pour que vous puissiez vous concentrer sur le CI/CD, pas sur le code métier. Mais les principes que vous apprendrez s'appliquent exactement de la même façon sur une vraie application en production.

Ce que fait l'application

Méthode HTTP	Route	Ce qu'elle fait	Exemple
GET	`/`	Message de bienvenue	`{"message": "Bienvenue !"}`
GET	`/health`	État de l'app (utilisé par les pipelines)	`{"status": "ok"}`
POST	`/add`	Addition	`{"a":3,"b":5}` → `{"result":8}`
POST	`/multiply`	Multiplication	`{"a":4,"b":6}` → `{"result":24}`

Structure des fichiers

```
cicdexo/
├── app/
│   ├── src/
│   │   ├── app.js          ← L'API Express (les routes)
│   │   └── server.js       ← Démarre le serveur sur le port 3000
│   ├── tests/
│   │   └── app.test.js     ← Tests qui vérifient que l'API fonctionne
│   ├── Dockerfile          ← Recette pour créer l'image Docker
│   ├── .eslintrc.json      ← Règles de style du code
│   └── package.json        ← Dépendances et scripts
│
├── .github/
│   └── workflows/
│       ├── ci.yml          ← Pipeline CI (lint, tests, sécu, docker)
│       ├── cd.yml          ← Pipeline CD (publish, staging, prod)
│       └── security-scan.yml ← Analyse sécurité hebdomadaire
│
└── docs/
    └── guide_cicd.html     ← Ce guide !
```

Technologies utilisées

Outil	Rôle	Étape du pipeline
ESLint	Analyse statique du code (conventions)	Lint
Jest + Supertest	Tests automatisés de l'API	Test
npm audit	Détecte les dépendances vulnérables	Security audit
Docker	Empaquette l'application	Build
Trivy	Scanner de vulnérabilités de l'image Docker	Build
CodeQL	Analyse de sécurité du code source (SAST)	Security scan
GHCR	GitHub Container Registry (stockage images Docker)	Publish

4. Étape 1 — Créer le dépôt GitHub

1 Créer un nouveau dépôt sur GitHub

1. Rendez-vous sur **github.com** et connectez-vous avec votre compte
2. Cliquez sur le bouton vert **"New"** en haut à gauche (ou le **+** en haut à droite → "New repository")
3. Remplissez le formulaire :
   - **Repository name** : `cicd-exercice`
   - **Description** : `Exercice CI/CD avec GitHub Actions`
   - **Visibility** : `Public` (pour accéder à toutes les fonctionnalités gratuitement)
   - **Ne cochez PAS** "Add a README file" — nous allons pousser notre propre code
4. Cliquez **"Create repository"**

2 Pousser le code sur GitHub

Ouvrez un terminal dans le dossier `cicdexo` et exécutez ces commandes **une par une** :

```
# 1. Initialiser git dans ce dossier
git init

# 2. Dire à git de surveiller tous les fichiers
git add .

# 3. Créer le premier commit (comme une sauvegarde nommée)
git commit -m "feat: ajout application et pipelines CI/CD"

# 4. Connecter ce dépôt local à GitHub
# IMPORTANT : remplacez youssef-esgi par VOTRE nom d'utilisateur GitHub
git remote add origin https://github.com/youssef-esgi/cicd-exercice.git

# 5. Renommer la branche principale en "main"
git branch -M main

# 6. Envoyer le code sur GitHub
git push -u origin main
```

💡 GitHub peut vous demander vos identifiants. Si vous avez activé la double authentification, vous devrez créer un "Personal Access Token" (Settings → Developer settings → Personal access tokens).

3 Vérifier que GitHub Actions est activé

1. Sur votre dépôt GitHub, cliquez sur l'onglet **"Actions"**
2. Si GitHub vous demande d'activer les Actions, cliquez **"I understand my workflows, go ahead and enable them"**
3. Vous devriez voir un workflow "CI - Intégration Continue" déjà en cours d'exécution (cercle orange) !

✅ Le simple fait de pousser le dossier `.github/workflows/` a automatiquement déclenché le pipeline CI. Pas besoin de cliquer sur "Run".

5. Étape 2 — Découvrir l'application

Avant d'analyser les pipelines, comprenons ce que fait l'application que nous allons déployer.

L'API (app/src/app.js)

L'application utilise **Express**, un framework web pour Node.js. Voici le code simplifié commenté :

```
const express = require('express');
const app = express();
app.use(express.json()); // Permet de lire le JSON des requêtes

// Route GET / → retourne un message de bienvenue
app.get('/', (req, res) => {
  res.json({ message: 'Bienvenue sur notre API !', version: '1.0.0' });
});

// Route POST /add → addition de deux nombres
app.post('/add', (req, res) => {
  const { a, b } = req.body; // On lit a et b dans le corps de la requête

  // Validation : on vérifie que a et b sont bien des nombres
  if (typeof a !== 'number' || typeof b !== 'number') {
    return res.status(400).json({ error: 'a et b doivent être des nombres' });
  }

  res.json({ result: a + b }); // On retourne le résultat
});
```

Les tests (app/tests/app.test.js)

Pour chaque route, on a des tests automatisés qui vérifient que tout fonctionne :

```
// Ce test vérifie que POST /add retourne bien 8 quand a=3 et b=5
test('additionne deux nombres', async () => {
  const res = await request(app).post('/add').send({ a: 3, b: 5 });
  expect(res.statusCode).toBe(200);  // Code HTTP 200 = succès
  expect(res.body.result).toBe(8);   // Le résultat doit être 8
});

// Ce test vérifie que l'API gère bien les erreurs
test('retourne une erreur si les paramètres sont invalides', async () => {
  const res = await request(app).post('/add').send({ a: 'foo', b: 5 });
  expect(res.statusCode).toBe(400);        // Code 400 = mauvaise requête
  expect(res.body.error).toBeDefined();    // Un message d'erreur doit être présent
});
```

Le Dockerfile

Le Dockerfile est la recette pour créer une image Docker de l'application. Voici les bonnes pratiques de sécurité appliquées :

```
# Image Alpine = image très légère (moins de logiciels = moins de failles)
FROM node:20-alpine

# SÉCURITÉ : créer un utilisateur non-root
# Si l'application est piratée, l'attaquant n'aura pas les droits root
RUN addgroup -S appgroup && adduser -S appuser -G appgroup

WORKDIR /app

# OPTIMISATION : copier package.json en premier
# Si le code change mais pas les dépendances, Docker réutilise le cache
COPY package*.json ./
RUN npm ci --only=production  # Production seulement (pas les outils de dev)

COPY src/ ./src/

# SÉCURITÉ : basculer sur l'utilisateur non-root avant de lancer l'app
USER appuser

EXPOSE 3000
CMD ["node", "src/server.js"]
```

⚠️ **Pourquoi ne pas tourner en root dans un container ?**

Si votre application a une vulnérabilité et qu'un attaquant prend le contrôle du processus, un utilisateur root dans le container peut modifier le système de fichiers, installer des outils malveillants, ou tenter de s'échapper du container. Un utilisateur non-root limite les dégâts.

6. Étape 3 — Comprendre la structure d'un pipeline GitHub Actions

Ouvrez le fichier `.github/workflows/ci.yml` et observons-le ensemble.

Le déclencheur

```
on:
  push:
    branches: [ "**" ]        # ** = toutes les branches
  pull_request:
    branches: [ main ]        # Les PR ciblant main
```

Cela signifie : "Lance ce pipeline à chaque push sur n'importe quelle branche, et à chaque Pull Request vers main".

Les 4 jobs du pipeline CI

```
jobs:
  lint:                        # Job 1 : analyse du code
    runs-on: ubuntu-latest
    steps: [...]

  test:
    needs: lint                # Job 2 : attend que lint soit OK
    runs-on: ubuntu-latest
    steps: [...]

  security-audit:
    needs: lint                # Job 3 : tourne en parallèle avec "test"
    runs-on: ubuntu-latest
    steps: [...]

  build-docker:
    needs: [test, security-audit]  # Job 4 : attend les 2 jobs précédents
    runs-on: ubuntu-latest
    steps: [...]
```

Le mot-clé `needs`

💡 **needs = dépendance entre jobs**

`needs: lint` signifie "n'exécute ce job que si le job `lint` a réussi".

`needs: [test, security-audit]` signifie "n'exécute ce job que si les deux ont réussi".

Le job de lint détaillé

```
lint:
  name: Analyse du code (ESLint)
  runs-on: ubuntu-latest

  steps:
    # Action qui télécharge votre code sur la machine virtuelle
    - name: Récupérer le code source
      uses: actions/checkout@v4

    # Action qui installe Node.js (avec cache pour aller plus vite)
    - name: Installer Node.js
      uses: actions/setup-node@v4
      with:
        node-version: '20'
        cache: 'npm'
        cache-dependency-path: app/package-lock.json

    # Commande shell classique
    - name: Installer les dépendances
      run: npm ci
      working-directory: app    # Exécuté dans le dossier app/

    - name: Lancer ESLint
      run: npm run lint
      working-directory: app
```

Que fait ESLint ?

ESLint analyse le code **sans l'exécuter** pour vérifier qu'il respecte les règles définies dans `.eslintrc.json`.

```
{
  "rules": {
    "no-unused-vars": "error",    // Variable déclarée mais jamais utilisée → ERREUR
    "eqeqeq": "error",            // Utiliser === au lieu de == → ERREUR
    "semi": ["error", "always"],  // Oubli du ; → ERREUR
    "no-console": "warn"          // console.log() → AVERTISSEMENT
  }
}
```

7. Étape 4 — Observer le premier pipeline CI

1 Ouvrir l'onglet Actions

1. Sur votre dépôt GitHub → cliquez sur l'onglet **"Actions"**
2. Vous voyez le workflow "CI - Intégration Continue" avec un cercle orange (en cours) ou vert (terminé)
3. Cliquez dessus pour voir le détail

2 Lire les logs de chaque job

Cliquez sur chaque job pour voir les logs détaillés. Voici ce que vous devriez voir :

Job "lint" :

```
Run npm run lint
  app/src/app.js
    ✓ 0 problems (0 errors, 0 warnings)
  ✓ 1 file linted, no issues found.
```

Job "test" :

```
PASS tests/app.test.js
  GET /
    ✓ retourne un message de bienvenue (23ms)
  GET /health
    ✓ retourne le statut ok (5ms)
  POST /add
    ✓ additionne deux nombres (8ms)
    ✓ retourne une erreur si les paramètres ne sont pas des nombres (4ms)
  POST /multiply
    ✓ multiplie deux nombres (3ms)
    ✓ retourne une erreur si les paramètres ne sont pas des nombres (3ms)

Tests:  6 passed, 6 total
Coverage: Lines 92%, Functions 100%, Branches 83%
```

Job "security-audit" :

```
Run npm audit --audit-level=high
found 0 vulnerabilities
```

3 Télécharger le rapport de couverture

En bas de la page du workflow, dans la section **"Artifacts"**, vous trouverez un fichier **"coverage-report"**. Téléchargez-le.

Ouvrez `index.html` dans votre navigateur. Vous verrez ligne par ligne quelles parties du code sont couvertes par les tests (en vert) et lesquelles ne le sont pas (en rouge).

8. Étape 5 — Provoquer un échec (et comprendre pourquoi c'est utile)

🎯 **Objectif pédagogique**

On va volontairement "casser" quelque chose pour voir comment le pipeline réagit et nous protège.

1 Créer une nouvelle branche

On ne travaille jamais directement sur `main`. On crée une branche pour chaque modification :

```
git checkout -b feature/test-echec
```

2 Introduire une erreur de lint dans app.js

Ouvrez `app/src/app.js` et ajoutez cette ligne juste après `const app = express();` :

```
var maVariable = "test"  // var est interdit + manque le ; + guillemets doubles
```

Cette ligne viole 3 règles ESLint.

3 Pousser et observer l'échec

```
git add app/src/app.js
git commit -m "test: introduction d'une erreur volontaire"
git push origin feature/test-echec
```

Dans l'onglet Actions, vous verrez le job "lint" échouer avec une croix rouge ❌. Les logs indiquent précisément le problème :

```
app/src/app.js
  2:5  error  Unexpected var, use let or const instead  no-var
  2:20 error  Strings must use singlequote              quotes
  2:26 error  Missing semicolon                         semi
✖ 3 problems (3 errors, 0 warnings)
npm ERR! Process exited with code 1
```

Conséquence : les jobs "test" et "build-docker" ne s'exécutent même pas. Le pipeline s'arrête dès la première étape qui échoue.

4 Corriger et revenir au vert

Supprimez la ligne ajoutée, puis :

```
git add app/src/app.js
git commit -m "fix: suppression de l'erreur de lint"
git push origin feature/test-echec
```

Observez le pipeline repasser au vert ✅.

5 BONUS : casser un test unitaire

Dans `app/src/app.js`, modifiez la route `/add` pour retourner `a - b` au lieu de `a + b`. Poussez et observez le job "test" échouer.

```
FAIL tests/app.test.js
  × additionne deux nombres
    Expected: 8
    Received: -2
```

Le pipeline vous a **empêché de déployer un bug en production**.

9. Étape 6 — Sécurité : les secrets GitHub

🚨 **Règle absolue : Ne jamais stocker de secrets dans le code**

Un mot de passe, une clé API, un token dans le code source c'est une faille de sécurité critique. L'historique Git conserve TOUT. Même si vous supprimez le fichier, le secret reste accessible via `git log`.

Des incidents réels

Incident	Conséquence
Clé AWS commités sur GitHub	Des robots scannent GitHub en permanence. Des cryptomineurs ont utilisé le compte en quelques minutes → facture de 50 000$
Token de base de données exposé	Fuite de données clients
Clé API d'un service de paiement	Fraudes sur le compte

La solution : GitHub Secrets

GitHub Secrets est un coffre-fort chiffré intégré à GitHub. Les valeurs sont stockées chiffrées, jamais affichées en clair (même dans les logs), et accessibles depuis les workflows via `${{ secrets.NOM_DU_SECRET }}`.

1 Créer un secret

1. Sur votre dépôt → **Settings** → **Secrets and variables** → **Actions**
2. Cliquez **"New repository secret"**
3. Créez les secrets suivants :

Nom	Valeur (exemple pour l'exercice)
`DATABASE_URL`	`postgresql://user:password@localhost/db`
`MON_SECRET`	`super-secret-123`

⚠️ Une fois créé, la valeur d'un secret ne peut plus être lue, même par vous. Vous pouvez seulement la remplacer ou la supprimer.

2 Utiliser un secret dans un pipeline

Dans un fichier workflow, on accède aux secrets avec `${{ secrets.NOM }}` :

```
- name: Déployer avec les credentials
  run: ./deploy.sh
  env:
    DATABASE_URL: ${{ secrets.DATABASE_URL }}  # Injecté comme variable d'environnement
```

GitHub remplace automatiquement `${{ secrets.DATABASE_URL }}` par la vraie valeur dans la machine virtuelle, et **masque** toute apparition de cette valeur dans les logs par `***`.

3 Test pratique : voir le masquage en action

Ajoutez ce job temporaire à `.github/workflows/ci.yml` :

```
test-secret:
  name: Test du masquage des secrets
  runs-on: ubuntu-latest
  steps:
    - name: Afficher le secret (masqué automatiquement)
      run: echo "La valeur est : ${{ secrets.MON_SECRET }}"
```

Poussez et observez les logs : vous verrez `La valeur est : ***`. La valeur n'est jamais exposée.

Le secret GITHUB_TOKEN

Dans le fichier `cd.yml`, vous remarquerez :

```
password: ${{ secrets.GITHUB_TOKEN }}
```

Ce token est **injecté automatiquement** par GitHub dans chaque workflow. Il permet de s'authentifier aux services GitHub (Container Registry, API GitHub) depuis un pipeline, sans que vous ayez rien à configurer. Il est valable uniquement pendant l'exécution du workflow.

10. Étape 7 — Protection des branches

Sans protection de branche, n'importe qui peut pousser du code directement sur `main`, sans que personne le relise, et sans que le CI tourne.

1 Configurer la protection de main

1. Sur votre dépôt → **Settings** → **Branches**
2. Cliquez **"Add branch protection rule"**
3. Dans "Branch name pattern", écrivez exactement : `main`
4. Cochez ces options :

Option	Ce que ça fait
✅ Require a pull request before merging	Impossible de pousser directement sur main — une PR est obligatoire
✅ Require status checks to pass	Le CI doit réussir avant de pouvoir merger
✅ Require branches to be up to date	La branche doit être à jour avec main avant le merge
✅ Do not allow bypassing	Même les admins du dépôt respectent les règles

5. Dans "Status checks", recherchez et ajoutez : `lint`, `test`, `security-audit`
6. Cliquez **"Save changes"**

2 Tester la protection

Essayez de pousser directement sur main :

```
git checkout main
echo "// test" >> app/src/app.js
git add . && git commit -m "test push direct"
git push origin main
```

Vous obtenez :

```
remote: error: GH006: Protected branch update failed for refs/heads/main.
remote: error: Required status check "test" is expected.
To https://github.com/youssef-esgi/cicd-exercice.git
 ! [remote rejected] main -> main (protected branch hook declined)
```

✅ GitHub refuse le push. Vous êtes obligé de passer par une Pull Request dont tous les checks CI doivent être verts.

11. Étape 8 — Le pipeline CD et les environnements

Comprendre les environnements

Un environnement en CI/CD est une "cible de déploiement" avec ses propres règles :

```
DEV STAGING PRODUCTION ─── ────────── ──────────────── Local Automatique
Approbation (votre après merge manuelle machine) dans main requise
```

1 Créer les environnements

1. Sur votre dépôt → **Settings** → **Environments**
2. Cliquez **"New environment"** → nommez-le `staging` → cliquez "Configure environment"
3. Cliquez **"New environment"** → nommez-le `production` → cliquez "Configure environment"
4. Pour `production` : cochez **"Required reviewers"** et ajoutez votre propre nom

Désormais, le déploiement en production est bloqué jusqu'à ce qu'un humain l'approuve manuellement.

2 Autoriser les workflows à écrire dans le Container Registry

1. Sur votre dépôt → **Settings** → **Actions** → **General**
2. Sous "Workflow permissions", sélectionnez **"Read and write permissions"**
3. Sauvegardez

3 Observer le pipeline CD

Mergez la Pull Request créée à l'étape 5 (ou créez-en une nouvelle). Dès que le merge est effectué sur main, le pipeline CD se déclenche :

1. **publish** : l'image Docker est construite et publiée sur `ghcr.io/votre-username/cicd-exercice`
2. **deploy-staging** : déploiement automatique en staging
3. **deploy-production** : ⏸️ EN ATTENTE — vous recevez un email !

Pour approuver le déploiement en production :

1. Allez dans l'onglet Actions → cliquez sur le workflow CD en cours
2. Cliquez sur le job **"deploy-production"**
3. Cliquez **"Review deployments"**
4. Cochez "production" et cliquez **"Approve and deploy"**

4 Voir l'image publiée

1. Allez sur votre profil GitHub (pas le dépôt, votre profil)
2. Cliquez sur l'onglet **"Packages"**
3. Vous voyez votre image `cicd-exercice` avec deux tags : `latest` et `sha-XXXXXXXX`

Depuis n'importe quelle machine, on peut maintenant télécharger cette image :

```
docker pull ghcr.io/youssef-esgi/cicd-exercice:latest
```

12. Étape 9 — Sécurité des images Docker avec Trivy

🔍 **C'est quoi Trivy ?**

Trivy (prononcé "try-vee") est un scanner de vulnérabilités open source créé par la société Aqua Security. Il analyse une image Docker couche par couche pour trouver des failles de sécurité connues dans les logiciels installés.

Pourquoi scanner une image Docker ?

Quand vous créez une image Docker avec `FROM node:20-alpine`, vous héritez de tout ce qui est dans cette image : l'OS Alpine Linux, le runtime Node.js, des dizaines de bibliothèques système... Ces composants peuvent contenir des **vulnérabilités connues**, identifiées par des numéros CVE (Common Vulnerabilities and Exposures).

```
Votre image Docker = couches empilées ┌──────────────────────────────────┐ │
Votre code (app.js, server.js) │ ← Analysé par CodeQL (SAST)
├──────────────────────────────────┤ │ Vos dépendances npm (express...) │ ←
Analysé par npm audit ET Trivy ├──────────────────────────────────┤ │ Runtime
Node.js 20 │ ← Analysé par Trivy ├──────────────────────────────────┤ │ OS
```

```
Alpine Linux (libc, openssl...) │ ← Analysé par Trivy
└──────────────────────────────────────────┘
```

npm audit vs Trivy : quelle différence ?

Outil	Ce qu'il analyse	Quand il tourne
npm audit	Les packages npm (express, jest, eslint...)	Job `security-audit`
Trivy	L'image Docker entière : OS + runtime Node.js + packages npm de production	Job `build-docker`

> ⚠️ **Un cas réel : Log4Shell (CVE-2021-44228)**
> En 2021, une vulnérabilité critique dans Log4j (bibliothèque Java) a affecté des millions de serveurs. Les équipes qui avaient Trivy dans leur pipeline ont pu détecter et corriger la faille en quelques heures. Celles qui ne scannaient pas leurs images l'ont découvert... après l'attaque.

Comment Trivy est intégré dans notre pipeline

Dans le job `build-docker` de `ci.yml`, Trivy est lancé juste après la construction de l'image :

```
# 1. Construire l'image Docker (disponible localement sur l'Ubuntu runner)
- name: Construire l'image Docker
  run: docker build -t cicd-exercice:${{ github.sha }} app/

# 2. Scanner l'image avec Trivy
- name: Scanner l'image Docker pour les vulnérabilités (Trivy)
  uses: aquasecurity/trivy-action@0.20.0   # Version fixe (bonne pratique sécurité)
  with:
    image-ref: cicd-exercice:${{ github.sha }}  # L'image qu'on vient de construire
    format: 'table'                             # Affichage en tableau dans les logs
    exit-code: '1'                              # Exit code 1 = pipeline échoue si vulnérable
    ignore-unfixed: true                        # Ignorer les CVE sans correctif disponible
    vuln-type: 'os,library'                     # Analyser l'OS et les bibliothèques
    severity: 'CRITICAL,HIGH'                   # Seulement les failles graves
```

Comprendre les niveaux de sévérité CVE

Niveau	Score CVSS	Description	Notre pipeline
CRITICAL	9.0 – 10.0	Exploitation facile, impact maximal (ex : exécution de code à distance)	❌ Pipeline bloqué
HIGH	7.0 – 8.9	Impact important (ex : vol de données, élévation de privilèges)	❌ Pipeline bloqué
MEDIUM	4.0 – 6.9	Impact modéré, conditions d'exploitation particulières	⚠️ Affiché mais ignoré
LOW	0.1 – 3.9	Impact faible	⚠️ Affiché mais ignoré

Exemple de rapport Trivy dans les logs

Voici à quoi ressemble un rapport Trivy sans vulnérabilités critiques (ce que vous devriez voir) :

```
2024-03-27T08:12:45Z INFO Vulnerability scanning is enabled
2024-03-27T08:12:52Z INFO Detected OS family="alpine" version="3.19.1"

cicd-exercice:abc1234 (alpine 3.19.1)
=====================================
Total: 0 (HIGH: 0, CRITICAL: 0)
```

Et un exemple avec une vulnérabilité HIGH détectée (pipeline qui échoue) :

```
cicd-exercice:abc1234 (alpine 3.19.1)
=====================================
Total: 1 (HIGH: 1, CRITICAL: 0)

┌─────────┬────────────────┬──────────┬──────────────────┬────────────────┐
│ Library │ Vulnerability  │ Severity │ Installed Version│ Fixed Version  │
├─────────┼────────────────┼──────────┼──────────────────┼────────────────┤
│ openssl │ CVE-2024-XXXX  │ HIGH     │ 3.1.4-r5         │ 3.1.5-r0       │
└─────────┴────────────────┴──────────┴──────────────────┴────────────────┘
```

Comment corriger ? Mettre à jour l'image de base dans le Dockerfile :

```
# FROM node:20-alpine   ← ancienne image
FROM node:20-alpine    ← GitHub Dependabot peut proposer des mises à jour automatiquement
```

Pourquoi Trivy scanne aussi les packages npm ?

Dans notre Dockerfile, on utilise `npm ci --only=production`, ce qui installe seulement **express** (pas jest, eslint, etc.). Trivy va donc scanner uniquement les dépendances de production dans l'image. Comparé à `npm audit` qui scanne aussi les devDependencies (jest, eslint...), Trivy nous donne une vue plus précise de la sécurité de ce qui tourne réellement en production.

💡 **Le résultat de npm audit de notre projet**

Si vous avez lancé `npm audit` en local, vous avez peut-être vu 26 vulnérabilités "moderate". Ce sont des vulnérabilités dans jest et eslint (devDependencies). Notre pipeline utilise `--audit-level=high`, donc le pipeline **ne bloque pas** sur les "moderate".

Et comme ces outils ne sont jamais dans l'image Docker finale (grâce à `--only=production`), Trivy ne les verra pas non plus. **Zéro vulnérabilité CRITICAL ou HIGH en production.**

1 Observer le scan Trivy dans les logs GitHub Actions

1. Allez dans l'onglet **Actions** de votre dépôt
2. Cliquez sur le dernier run du workflow "CI - Intégration Continue"
3. Cliquez sur le job **"Construction image Docker"**
4. Développez l'étape **"Scanner l'image Docker pour les vulnérabilités (Trivy)"**
5. Vous devriez voir : `Total: 0 (HIGH: 0, CRITICAL: 0)`

✅ Notre image basée sur `node:20-alpine` est régulièrement maintenue par l'équipe Node.js officielle. Elle ne contient pas de vulnérabilités HIGH ou CRITICAL connues.

13. Étape 10 — Analyse du code source avec CodeQL (SAST)

Qu'est-ce que le SAST ?

SAST signifie **Static Application Security Testing** : analyser le code source **sans l'exécuter** pour détecter des vulnérabilités.
CodeQL est l'outil SAST de GitHub. Il comprend la sémantique du code (pas juste des expressions régulières) et peut détecter des failles complexes.

Vulnérabilité détectée par CodeQL	Description
Path Traversal	Accéder à des fichiers hors du dossier prévu via `../../`
SQL Injection	Données utilisateur insérées directement dans une requête SQL
XSS (Cross-Site Scripting)	Données utilisateur insérées dans du HTML sans nettoyage
Command Injection	Données utilisateur passées à une commande shell
Mauvaise gestion des secrets	Credentials codés en dur dans le code

1 Activer l'analyse CodeQL

1. Sur votre dépôt → **Settings** → **Code security and analysis**
2. Activez **"Code scanning"** (cliquez sur "Set up" → "Default")
3. Le fichier `security-scan.yml` que vous avez déjà dans `.github/workflows/` tournera automatiquement

2 Simuler une vulnérabilité (exercice démonstration)

⚠️ Ceci est un exercice pédagogique. Ne jamais mettre ce type de code en production.

Créez une branche et ajoutez cette route vulnérable à `app.js` :

```
const fs = require('fs');
const path = require('path');

// VULNÉRABILITÉ INTENTIONNELLE : Path Traversal
app.get('/read-file', (req, res) => {
  // DANGER : l'utilisateur contrôle le chemin du fichier !
  const filePath = path.join(__dirname, req.query.filename);
  const content = fs.readFileSync(filePath, 'utf8');
  res.send(content);
});
```

Un attaquant pourrait appeler :
`GET /read-file?filename=../../../../etc/passwd`
Et lire le fichier des utilisateurs du système !
Poussez ce code et attendez l'analyse CodeQL (peut prendre quelques minutes). Une alerte apparaîtra dans **Onglet Security → Code scanning alerts**.

3 Corriger la vulnérabilité

Voici la version corrigée :

```
app.get('/read-file', (req, res) => {
  const allowedDir = path.join(__dirname, 'public-files');
  const filePath = path.join(allowedDir, req.query.filename);

  // Vérification : le chemin résolu doit rester dans le dossier autorisé
  if (!filePath.startsWith(allowedDir)) {
    return res.status(403).json({ error: 'Accès refusé' });
  }

  const content = fs.readFileSync(filePath, 'utf8');
  res.send(content);
});
```

Maintenant, même si l'utilisateur essaie `../../etc/passwd`, le chemin résolu sort du dossier autorisé et la requête est rejetée avec une erreur 403.

14. Étape 11 — Scénario complet : la Pull Request

On va simuler un workflow d'équipe réel, de la feature branch jusqu'au déploiement en production.

1 Créer la feature branch

```
git checkout -b feature/division
```

2 Ajouter la route /divide dans app.js

Dans `app/src/app.js`, ajoutez avant `module.exports = app;` :

```
app.post('/divide', (req, res) => {
  const { a, b } = req.body;
  if (typeof a !== 'number' || typeof b !== 'number') {
    return res.status(400).json({ error: 'Les paramètres a et b doivent être des nombres' });
  }
  if (b === 0) {
    return res.status(400).json({ error: 'Division par zéro impossible' });
  }
  res.json({ result: a / b });
});
```

3 Ajouter les tests dans app.test.js

Dans `app/tests/app.test.js`, ajoutez :

```
describe('POST /divide', () => {
  test('divise deux nombres', async () => {
    const res = await request(app).post('/divide').send({ a: 10, b: 2 });
    expect(res.statusCode).toBe(200);
    expect(res.body.result).toBe(5);
  });

  test('retourne une erreur pour une division par zéro', async () => {
    const res = await request(app).post('/divide').send({ a: 10, b: 0 });
    expect(res.statusCode).toBe(400);
    expect(res.body.error).toBe('Division par zéro impossible');
  });
});
```

4 Pousser et créer la Pull Request

```
git add .
git commit -m "feat: ajout de la route /divide avec gestion division par zéro"
git push origin feature/division
```

Sur GitHub, une bannière jaune apparaît en haut du dépôt :
"feature/division had recent pushes less than a minute ago. Compare & pull request"
Cliquez sur **"Compare & pull request"**.
Remplissez la PR :

- **Title** : `feat: ajout de la division`
- **Description** :

```
## Ce que j'ai fait
- Ajout de la route POST /divide
- Gestion de la division par zéro (retourne 400)
- Tests unitaires associés

## Comment tester
curl -X POST http://localhost:3000/divide \
  -H "Content-Type: application/json" \
  -d '{"a":10,"b":2}'
# Réponse attendue : {"result":5}
```

Cliquez **"Create pull request"**.

5 Observer le CI sur la PR

Sur la page de la PR, vous voyez les checks CI en temps réel :

```
Some checks haven't completed yet
  ● lint — in progress...

All checks have passed
  ✅ lint — passed
  ✅ test — passed
  ✅ security-audit — passed
  ✅ build-docker — passed
```

Le bouton **"Merge pull request"** ne devient cliquable que quand tous les checks sont verts.

6 Merger et suivre le CD

1. Cliquez **"Merge pull request"** → **"Confirm merge"**
2. Allez dans l'onglet **Actions** → vous voyez le pipeline CD se lancer automatiquement
3. L'image Docker avec la nouvelle route `/divide` est publiée automatiquement
4. Approuvez le déploiement en production quand GitHub vous le demande

🎉 **Félicitations !**

Vous venez de réaliser un cycle CI/CD complet : feature branch → PR → CI → merge → CD → production. C'est exactement le workflow utilisé dans les équipes professionnelles.

15. Récapitulatif : les 7 règles d'or de la sécurité

CI/CD

1 Épingler les versions des actions tierces

```
# ❌ Dangereux : si quelqu'un compromet le dépôt de l'action, votre pipeline est compromis
- uses: actions/checkout@main

# ✅ Sécurisé : hash de commit immuable (impossible à falsifier)
- uses: actions/checkout@11bd71901bbe5b1630ceea73d27597364c9af683  # v4.2.2
```

2 Principe du moindre privilège

```
# Donner seulement les permissions nécessaires au pipeline
permissions:
  contents: read       # Peut lire le code
  packages: write      # Peut publier des images Docker
  # Pas de permissions d'écriture sur le code, pas d'accès aux secrets d'autres repo
```

3 Jamais de secrets dans le code

```
# ❌ CATASTROPHIQUE
DATABASE_URL = "postgresql://admin:motdepasse123@prod-server/db"  # Dans le code sou

# ✅ CORRECT
DATABASE_URL: ${{ secrets.DATABASE_URL }}  # Via GitHub Secrets
```

4 Revue de code obligatoire (Pull Requests)

La protection de branche + les PR obligatoires garantissent que chaque modification est relue par un autre humain avant d'aller en production.

5 Scanner les dépendances régulièrement

Les bibliothèques open source sont une surface d'attaque majeure. Une dépendance peut être vulnérable des mois après sa publication.

- `npm audit` : vérifie les CVE connues
- Dependabot : crée automatiquement des PRs pour mettre à jour les dépendances
- Trivy : scanner les images Docker

6 Environnements séparés + approbation manuelle pour la prod

```
feature branch ⟶ staging (automatique) ⟶ production
(approbation humaine requise)
```

7 Ne pas tourner en root dans les containers

Si l'application est compromise, un utilisateur non-root limite considérablement ce que l'attaquant peut faire dans le container.

16. Exercices bonus

Bonus 1 — Ajouter Dependabot (mise à jour automatique des dépendances)

Créez le fichier `.github/dependabot.yml` :

```
version: 2
updates:
  - package-ecosystem: "npm"
    directory: "/app"
    schedule:
      interval: "weekly"    # Vérifie chaque semaine les mises à jour
```

Dependabot va créer automatiquement des Pull Requests pour mettre à jour vos dépendances quand de nouvelles versions sortent.

Bonus 2 — Matrice de tests (plusieurs versions de Node)

Modifiez le job `test` dans `ci.yml` :

```
test:
  strategy:
    matrix:
      node-version: [18, 20, 22]  # 3 jobs lancés en parallèle
  name: Tests (Node ${{ matrix.node-version }})
  runs-on: ubuntu-latest
  steps:
    - uses: actions/setup-node@v4
      with:
        node-version: ${{ matrix.node-version }}
```

Bonus 3 — Notification Slack en cas de déploiement réussi

1. Créez un webhook entrant dans votre espace Slack
2. Ajoutez-le comme secret : `SLACK_WEBHOOK_URL`

3. Ajoutez cette étape à la fin du job `deploy-production` :

```
- name: Notifier Slack
  if: success()
  run: |
    curl -X POST -H 'Content-type: application/json' \
      --data "{\"text\":\"Déploiement réussi en production ! Commit: ${{ github.sha }}\"}" \
      ${{ secrets.SLACK_WEBHOOK_URL }}
```

Bonus 4 — Badge de statut dans le README

Ajoutez ce badge dans votre `README.md` pour afficher le statut du CI :

```
![CI](https://github.com/youssef-esgi/cicd-exercice/actions/workflows/ci.yml/badge.svg)
```

Il affichera un badge vert "passing" ou rouge "failing" selon le dernier run du pipeline.

🏆 **Vous avez terminé l'exercice !**

Vous savez maintenant mettre en place un pipeline CI/CD complet et sécurisé avec GitHub Actions. Ces compétences sont directement applicables dans un contexte professionnel DevSecOps.

Ce que vous maîtrisez maintenant :

- Créer et lire un fichier workflow GitHub Actions
- Configurer du lint, des tests et un audit de sécurité automatiques
- Construire et publier une image Docker sécurisée
- Gérer les secrets sans jamais les exposer
- Protéger les branches et imposer les PR
- Mettre en place des environnements avec approbation manuelle
- Analyser la sécurité du code avec CodeQL